

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: pf	Course Code: CMPE-112L
Assignment Type: lab	Dated: 18-05-2026
Semester: 1	Session: Spring 2026
Lab/Project/Assignment #: 11	CLOs to be covered: CLO 4
Lab Title: Exception handling	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals/Objectives:

- To understand the concept of exception handling in Python.
- To learn the use of `try`, `except`, `else`, and `finally` blocks.
- To handle runtime errors and prevent program crashes.
- To develop robust and reliable Python programs using exception handling techniques

Equipment Required:

- Laptop/PC
- Python 3.x
- VS Code (or any Python IDE)
- Windows Operating System

Theory:

Exception handling is a mechanism in Python that allows a program to detect and handle runtime errors without terminating unexpectedly. The `try` block contains the code that may raise an exception, while the `except` block handles the error. The `else` block executes if no exception occurs, and the `finally` block is executed regardless of whether an exception occurs or not. Exception handling improves the reliability, readability, and stability of programs by allowing developers to manage unexpected situations gracefully.

Lab Work:

Task 1:

Write a program to calculate the factorial of a number using exception handling. Use `try` and `except` blocks to handle invalid input and an `assert` statement to ensure that the entered number is non-negative.

Result:

```

5      # Validate that the number is non-negative
6      assert num >= 0, "Negative numbers are not allowed."
7
8      # Calculate factorial
9      factorial = 1
10     for i in range(1, num + 1):
11         factorial *= i
12
13     print(f"Factorial of {num} is {factorial}")
14
15 except ValueError:
16     print("Error: Please enter a valid integer, not a string.")
17
18 except AssertionError as e:
19     print("Error:", e)

```

Terminal Output:

```

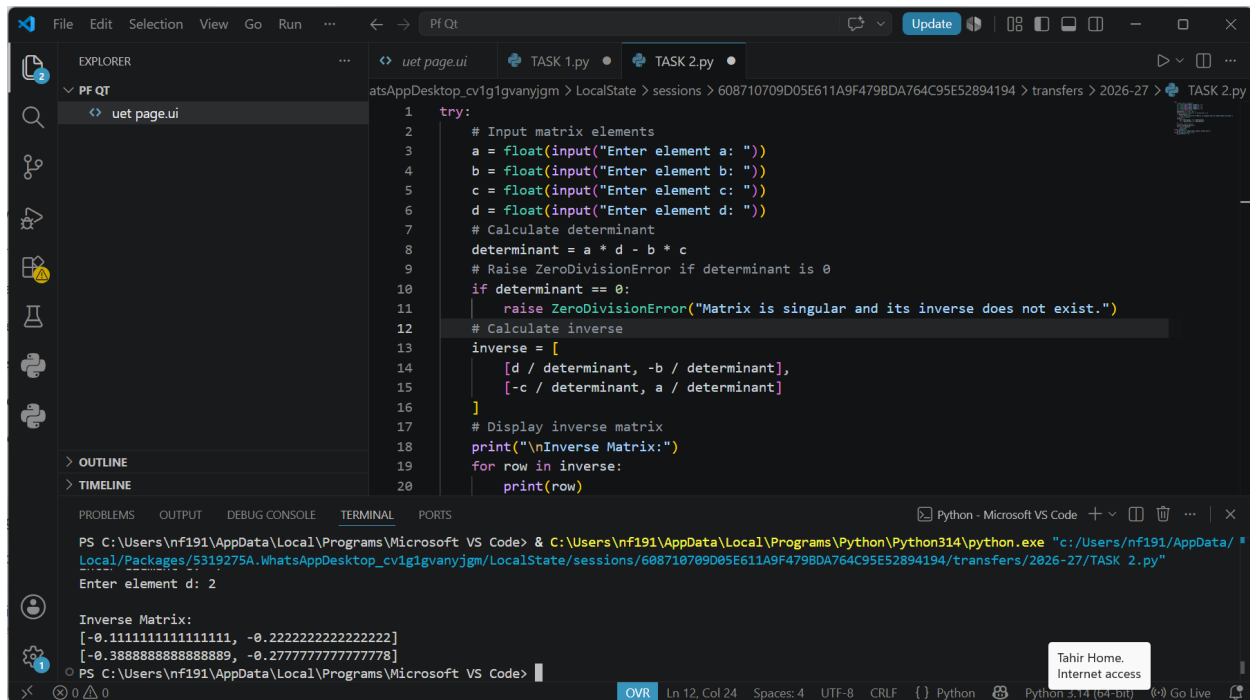
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/AppData/Local/Packages/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F479BDA764C95E52894194/transfers/2026-27/TASK 1.py"
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code>

```

Task 2:

Write a program to calculate the inverse of a 2×2 matrix using exception handling. Use `try` and `except` blocks to handle invalid input and raise an exception if the matrix is singular (determinant equals zero).

Result:



```
1 try:
2     # Input matrix elements
3     a = float(input("Enter element a: "))
4     b = float(input("Enter element b: "))
5     c = float(input("Enter element c: "))
6     d = float(input("Enter element d: "))
7     # Calculate determinant
8     determinant = a * d - b * c
9     # Raise ZeroDivisionError if determinant is 0
10    if determinant == 0:
11        raise ZeroDivisionError("Matrix is singular and its inverse does not exist.")
12    # Calculate inverse
13    inverse = [
14        [d / determinant, -b / determinant],
15        [-c / determinant, a / determinant]
16    ]
17    # Display inverse matrix
18    print("\nInverse Matrix:")
19    for row in inverse:
20        print(row)
```

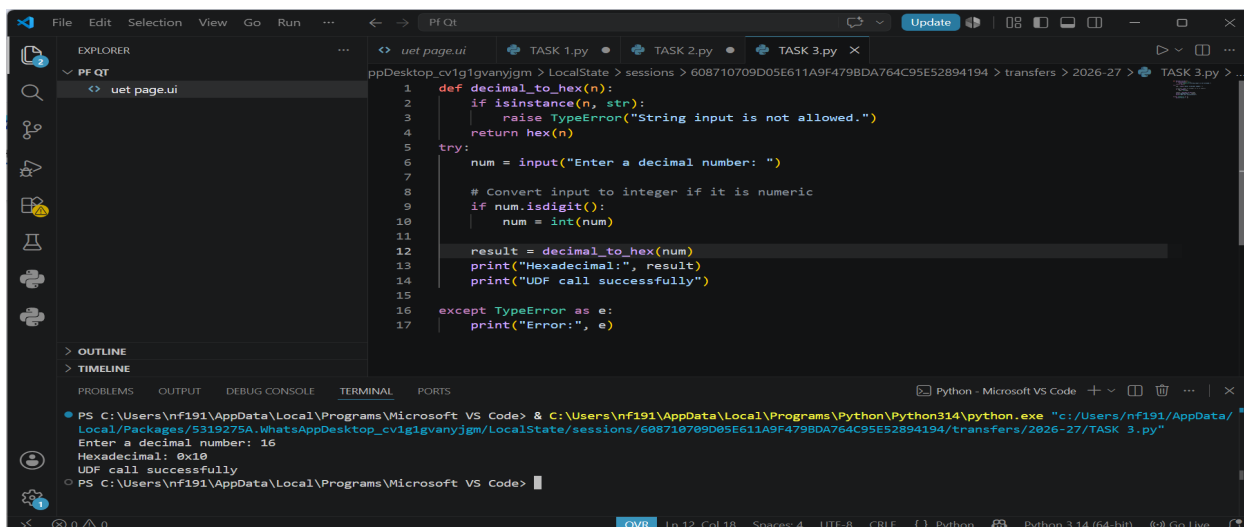
Enter element d: 2

Inverse Matrix:
[-0.1111111111111111, -0.2222222222222222]
[-0.3888888888888889, -0.2777777777777778]

Task 3:

Write a program to convert a decimal number to hexadecimal using a user-defined function and exception handling. Raise a `TypeError` if the input is a string and display an appropriate error message.

Result:



```
1 def decimal_to_hex(n):
2     if isinstance(n, str):
3         raise TypeError("String input is not allowed.")
4     return hex(n)
5
6 try:
7     num = input("Enter a decimal number: ")
8     # Convert input to integer if it is numeric
9     if num.isdigit():
10        num = int(num)
11
12    result = decimal_to_hex(num)
13    print("Hexadecimal:", result)
14    print("UDF call successfully")
15
16 except TypeError as e:
17    print("Error:", e)
```

Enter a decimal number: 16

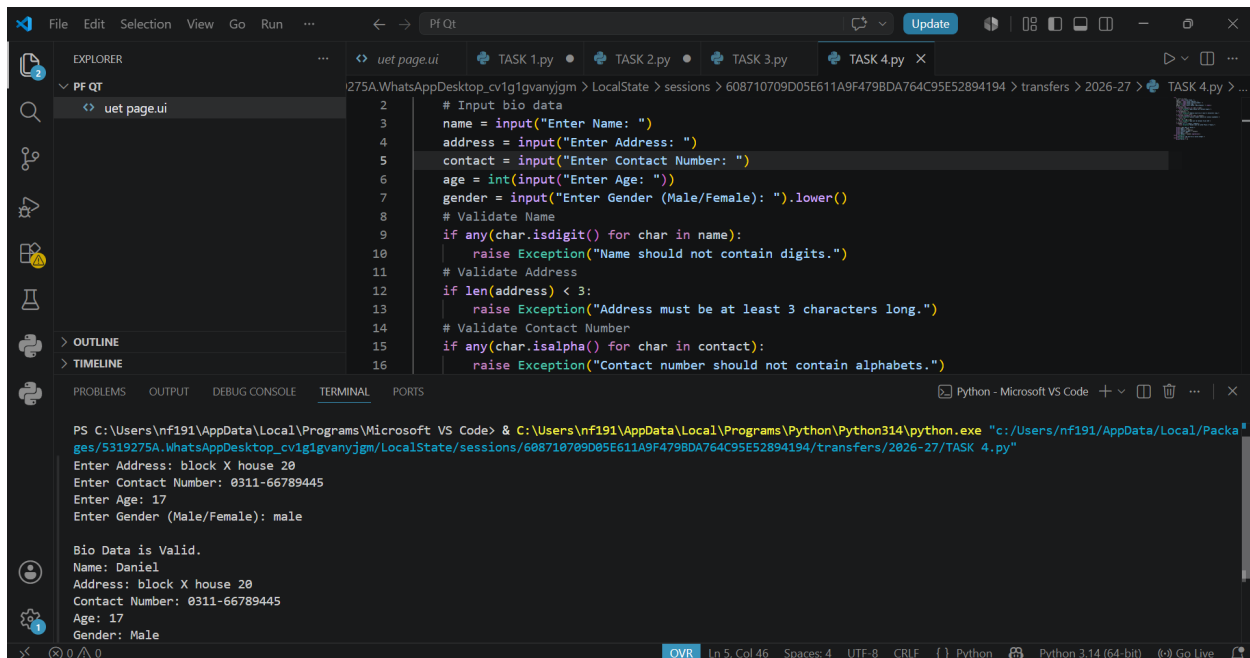
Hexadecimal: 0x10

UDF call successfully

Task 4:

Write a program to validate user bio-data using exception handling. Check the validity of the name, address, contact number, age, and gender, and display appropriate error messages for invalid input.

Result:



```
275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > 608710709D05E611A9F4798DA764C95E52894194 > transfers > 2026-27 > TASK 4.py > ...
2  # Input bio data
3  name = input("Enter Name: ")
4  address = input("Enter Address: ")
5  contact = input("Enter Contact Number: ")
6  age = int(input("Enter Age: "))
7  gender = input("Enter Gender (Male/Female): ").lower()
8  # Validate Name
9  if any(char.isdigit() for char in name):
10     raise Exception("Name should not contain digits.")
11  # Validate Address
12  if len(address) < 3:
13     raise Exception("Address must be at least 3 characters long.")
14  # Validate Contact Number
15  if any(char.isalpha() for char in contact):
16     raise Exception("Contact number should not contain alphabets.")

PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/AppData/Local/PackageS519275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F4798DA764C95E52894194/transfers/2026-27/TASK 4.py"
Enter Address: block X house 20
Enter Contact Number: 0311-66789445
Enter Age: 17
Enter Gender (Male/Female): male

Bio Data is Valid.
Name: Daniel
Address: block X house 20
Contact Number: 0311-66789445
Age: 17
Gender: Male
```

Conclusion:

In this lab, we learned how to use exception handling in Python to detect and manage runtime errors effectively. We practiced using `try`, `except`, and `raise` statements to handle different types of exceptions, validate user input, and display meaningful error messages. We also implemented exception handling in practical programs such as factorial calculation, matrix inverse computation, decimal-to-hexadecimal conversion, and bio-data validation. This lab improved our understanding of writing robust, reliable, and user-friendly Python programs.